

DOSSIER TECHNIQUE DE RÉFÉRENCE

EduTutor

Tuteur académique multi-agents propulsé par Hermes Agent
Architecture, décisions d'ingénierie et défense du projet

Pile technique : React + Vite · FastAPI · ChromaDB · sentence-transformers
(embedding multilingue + cross-encoder) · Hermes Agent CLI → Claude.

Couches : RAG (avec vision des schémas) · système multi-agents à routage
déterministe · application web multi-utilisateur avec streaming temps réel (SSE).

Fils conducteurs : traçabilité · robustesse (aucune panne silencieuse) · isolation
par construction · coût maîtrisé.

Sommaire

1. Résumé exécutif
2. Vision produit et positionnement
3. Vue d'ensemble de l'architecture
4. Les décisions d'ingénierie (et pourquoi)
5. Le moteur d'inférence : Hermes Agent
6. Le RAG, étage par étage
7. Le système multi-agents
8. Multi-utilisateur et sécurité
9. Le backend API (FastAPI + SSE)
10. Le frontend (React)
11. Persistance, identité et caches
12. Robustesse et gestion d'erreurs
13. Stratégie de test
14. Parcours d'une requête, de bout en bout
15. Référence fichier par fichier
16. Limites connues et évolutions
17. Glossaire

1. Résumé exécutif

EduTutor (nom de code *Hermes Education*) est une plateforme de tutorat académique. Un étudiant y dépose ses cours (PDF, Word, PowerPoint, images), pose des questions, demande des résumés, des fiches de révision ou la rédaction de documents originaux. Les réponses sont **ancrées sur ses propres cours** grâce à un moteur de recherche sémantique (RAG), et produites par un **modèle de langage de pointe** (Claude, via le framework Hermes Agent).

Le système repose sur **trois couches nettement séparées** :

- un **RAG** (Retrieval-Augmented Generation) qui indexe les cours et retrouve les passages pertinents, y compris le contenu des **schémas et figures** (analyse visuelle) ;
- un **système multi-agents** orchestré de façon **déterministe** (tuteur, analyse documentaire, génération de contenu, rédaction) ;
- une **application web** (React + FastAPI) multi-utilisateur, avec **streaming temps réel** de la progression des agents.

Les fils conducteurs de la conception sont : **traçabilité** (on sait toujours ce qui est extrait par la machine vs inféré par le modèle), **robustesse** (aucune panne du modèle ne casse silencieusement l'application), **isolation** (chaque utilisateur est cloisonné par construction) et **coût maîtrisé** (petits

modèles rapides pour les tâches utilitaires, caches partout).

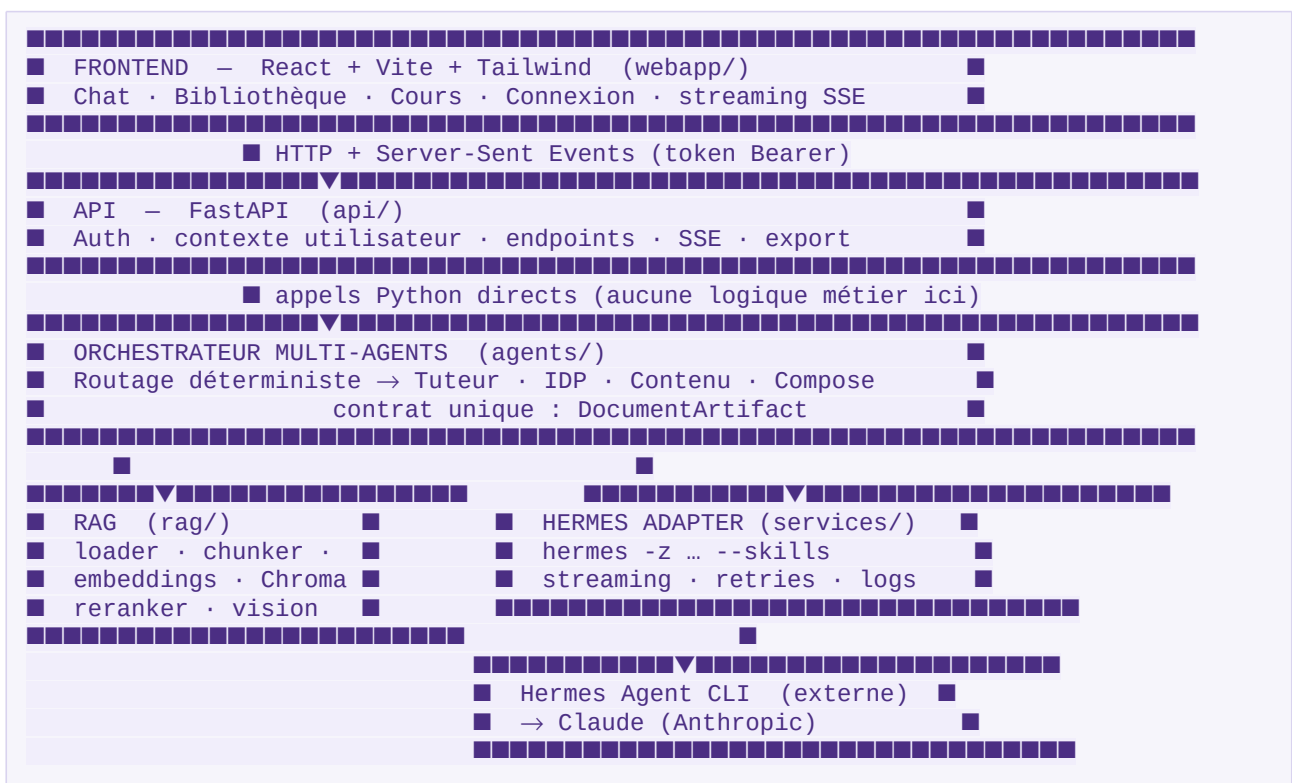
2. Vision produit et positionnement

EduTutor n'est **pas** un assistant documentaire générique. C'est un **tuteur** : les passages de cours servent de support, mais l'objectif est l'**accompagnement pédagogique**. Cette posture se traduit concrètement dans le code :

- le tuteur se présente comme *EduTutor* et relie chacune de ses capacités à l'aide à l'apprentissage (comprendre un cours, s'entraîner, produire un support), jamais comme un « assistant IA polyvalent » ;
- pour une question liée au cours, la réponse suit une **structure pédagogique imposée** en cinq points : *Réponse* · *Explication* · *Exemple* · *Indication de la partie du cours* · *Question de vérification* ;
- le système **reste honnête** : quand un passage n'est pas dans les cours, il le dit ; il ne fabrique pas de fausses références.

Le produit se compare, côté expérience, à ChatGPT/Claude (chat, historique, titres automatiques de conversation, livrables téléchargeables), mais **spécialisé** sur le corpus de l'étudiant et **transparent** sur ses sources.

3. Vue d'ensemble de l'architecture



Principe cardinal : l'API ne contient **aucune logique métier**. Elle est une façade HTTP qui appelle exactement la même couche que l'interface Streamlit historique (aujourd'hui archivée). Cette discipline

a permis de remplacer complètement le front sans toucher au cœur.

Trois couches, jamais fusionnées : le RAG ignore les agents, les agents ignorent HTTP, le front ignore Hermes. Chaque couche est testable seule.

4. Les décisions d'ingénierie (et pourquoi)

Cette section est le cœur de la « défense » du projet : pour chaque choix structurant, l'alternative envisagée et la raison du choix retenu.

4.1 ChromaDB plutôt que PostgreSQL/pgvector, Pinecone ou FAISS

Choix : ChromaDB (base vectorielle embarquée, persistée sur disque).

Alternative	Pourquoi écartée ici
PostgreSQL + pgvector	Impose un serveur de base de données à installer, administrer et sauvegarder. Surdimensionné pour un poste local/mono-serveur ; ajoute une dépendance opérationnelle lourde pour un gain nul à cette échelle.
Pinecone / service cloud	Externalise les données de cours vers un tiers (confidentialité), coûte de l'argent, exige une connexion réseau permanente et une clé. Incompatible avec la promesse « les cours de l'étudiant restent chez lui ».
FAISS brut	Bibliothèque d'index seulement : il faudrait recoder la persistance, les métadonnées, la fonction d'embedding, le filtrage. ChromaDB apporte tout cela intégré.

Raisons du choix ChromaDB : zéro serveur (client persistant sur le système de fichiers, `PersistentClient(path=...)`), métadonnées riches par chunk (source, section, page, mtime, `kind`), fonction d'embedding branchable, filtrage `where` natif (utilisé pour supprimer les figures d'un cours : `{"$and": [{"source": "..."}, {"kind": "image"}]}`), et surtout **une collection par utilisateur** (voir §8) qui donne une isolation *par construction*. La base vit dans `data/vectorstore/`.

4.2 Fichiers JSON plutôt qu'une base de données pour l'état applicatif

Choix : persistance en fichiers JSON (utilisateurs, conversations, artefacts, contenus générés, cache vision).

L'application est **mono-serveur, à faible concurrence** (un étudiant à la fois par compte). Introduire une base relationnelle pour stocker des conversations et des artefacts ajouterait un point de défaillance et de la complexité sans bénéfice. Les fichiers JSON sont : lisibles et inspectables à la main,

versionnables, triviaux à sauvegarder (copie de dossier), et sérialisés par un **verrou process-local** (`threading.Lock`) qui suffit en mono-process. Les écritures sont **best-effort** : une erreur d'écriture ne fait jamais échouer une requête (« la persistance ne doit jamais faire échouer une requête »).

Limite assumée : ce choix ne passe pas à l'échelle multi-process/multi-serveur ; c'est un compromis explicite d'un MVP mono-serveur, documenté comme tel dans le code.

4.3 Hermes Agent en CLI *one-shot* plutôt qu'un appel API direct au modèle

Choix : appeler le binaire `hermes -z "<prompt>" --skills <skill>` en sous-processus.

C'est la décision la plus contre-intuitive, et elle est délibérée. L'adaptateur (`services/hermes_adapter.py`) **n'importe jamais** le runtime Hermes et **ne fait jamais** d'appel LLM direct (OpenRouter/Anthropic) en remplacement silencieux.

Pourquoi passer par la CLI plutôt que par l'API du fournisseur ?

- **Les skills Hermes** (`education-tutor`, `education-idp`, `education-content`, `education-compose`, `education-orchestrator`) encapsulent les personas, les formats de sortie et les garde-fous. Les recharger via `--skills` garantit un comportement identique à celui validé en interactif.
- **Les outils** (vision, etc.) sont auto-approuvés et orchestrés par Hermes ; l'application n'a pas à réimplémenter la boucle d'agent.
- **Découplage** : le code applicatif ne dépend pas des versions internes du framework. Si Hermes évolue, l'interface `hermes -z` reste stable (c'est le point d'entrée officiel pour les scripts).
- **Sécurité de l'invocation** : la commande est passée en **liste d'arguments** (pas de shell), donc pas d'injection possible via le prompt.

Le mode `-z/--oneshot` imprime **uniquement** la réponse finale sur `stdout` (pas de bannière, pas de spinner), ce qui rend la sortie exploitable programmatiquement.

4.4 Routage déterministe plutôt qu'un « routeur LLM » pur

Choix : une table d'intention déterministe (expressions régulières) décide quels agents lancer ; le planner LLM n'est qu'une option.

Un système multi-agents « à la mode » laisserait un LLM décider seul du plan. Ici, l'orchestrateur (`agents/orchestrator.py`) est un **workflow défendable** :

- le routage **primaire** est une table d'intention traçable, indépendante du modèle et **testable sans aucun appel réseau** ;
- Python est le **seul exécuteur** : il insère lui-même les étapes manquantes (ex. une analyse IDP avant une génération de contenu) via `_repair_preconditions` ;
- le **planner LLM** (`use_planner=True`, skill `education-orchestrator`) reste disponible, mais son plan est **revalidé** côté Python (agents inconnus filtrés, préconditions vérifiées, documents résolus) ; s'il échoue, on **retombe** sur le routage déterministe.

Bénéfice : le comportement est **prévisible, reproductible et débogable**, et il fonctionne même quand le fournisseur LLM est indisponible pour la partie routage.

4.5 Récupération en deux étages : bi-encoder puis cross-encoder (reranker)

Choix : ChromaDB récupère un large vivier (`RETRIEVAL_TOP_K = 20`), un cross-encoder reclasse et on ne garde que `CONTEXT_TOP_K = 5`.

L'embedding (bi-encoder) encode la question et les chunks **séparément** : rapide mais approximatif. Le **cross-encoder** lit la paire (*question, chunk*) **ensemble** et produit un score de pertinence bien plus fin. On l'applique seulement au petit vivier (20 candidats), ce qui donne le meilleur rapport qualité/latence. Le reranker est **facultatif et dégradant proprement** : s'il est indisponible (hors-ligne, modèle absent, désactivé par `RERANKER_ENABLED=0`), on retombe sur le tri par distance sans planter.

4.6 Embedding multilingue

Choix : `paraphrase-multilingual-MiniLM-L12-v2`, distance cosinus.

Les cours de l'étudiant sont souvent en **anglais**, ses questions en **français**. Un embedding multilingue rapproche une question FR d'un passage EN (et inversement) dans le même espace vectoriel. La distance **cosinus** donne des valeurs stables dans `[0, 2]`, ce qui permet de **calibrer des seuils** de mode pédagogique (voir §7.5). Le modèle d'embedding est **centralisé** (`rag/embeddings.py`) : indexation et recherche utilisent forcément la même fonction, sinon les vecteurs ne seraient pas comparables.

4.7 Découpage *structure-aware* plutôt que « tous les N caractères »

Choix : découper sur les titres/sections détectés, puis empaqueter à ~900 caractères, sans jamais fusionner deux sections.

Un découpage naïf (fenêtre glissante aveugle) coupe au milieu des idées et mélange des sections sans rapport dans un même chunk. Le chargeur (`rag/document_loader.py`) détecte les titres par **cumul de signaux** (numérotation `1.2.3`, mots-clés `Chapitre/Section`, police plus grande/grasse dans les PDF, majuscules typographiques, titres Markdown), car **aucun signal n'est fiable seul**. Résultat : des chunks cohérents, et une **métadonnée section** qui alimente les « indications de cours » montrées à l'étudiant (« revois le cours X, partie Y »).

4.8 Le contrat de données unique `DocumentArtifact`

Choix : un objet structuré commun circule entre les agents, avec deux blocs strictement séparés.

- **extraction** : **déterministe** (Python, zéro LLM) — structure, pages, langue, qualité, **et le texte de chaque section**. C'est la « vérité machine ».
- **analysis** : **enrichi par le LLM** (type, thèmes, objectifs, définitions, consignes, dates), chaque élément **ancré à un `section_id` réel** avec un extrait de preuve.

Cette séparation garantit la **traçabilité** (on distingue toujours l'extrait de l'inféré) et la **provenance** (tout élément enrichi pointe vers une section réelle ; les `section_id` inventés par le modèle sont **filtrés**). L'agent de contenu ne relit **jamais** le PDF brut : il lit le texte **depuis l'artefact** (le contrat).

4.9 Sessions par token HMAC *stateless* plutôt que sessions serveur ou JWT

Choix : `token = base64url(user_id ++ HMAC-SHA256(user_id))`.

Alternative	Pourquoi écartée
Sessions serveur (table de sessions)	Impose un stockage à maintenir et invalide les sessions au redémarrage.
Bibliothèque JWT	Dépendance supplémentaire, surface d'attaque (algorithmes, <code>alg:none</code>), complexité inutile pour un besoin minimal.

Le token signé HMAC est **sans stockage serveur** (rien à persister), **survit au redémarrage** de l'API (le secret est persisté une fois dans `data/.session_secret`, en `0600`), et **ne dépend d'aucune bibliothèque externe**. Détail d'implémentation qui a coûté un vrai bug puis sa correction : la signature de 32 octets est concaténée **sans séparateur** et relue par **longueur fixe** (`raw[: -32]` / `raw[-32:]`) — utiliser un séparateur `.` était bogué, car un octet de la signature binaire peut être un `.`.

Limite assumée : pas de révocation fine ; changer le secret invalide **tous** les tokens (acceptable en mono-serveur).

4.10 Hachage de mot de passe PBKDF2-HMAC-SHA256

Choix : PBKDF2, 200 000 itérations, sel de 16 octets par compte.

PBKDF2 est dans la **bibliothèque standard** Python (`hashlib`), donc zéro dépendance. 200 000 itérations rendent la force brute coûteuse ; le sel par compte empêche les tables précalculées ; la comparaison se fait en **temps constant** (`hmac.compare_digest`). Le clair n'est **jamais** stocké, et la vue publique d'un utilisateur n'expose **jamais** le hash ni le sel. (*bcrypt/argon2 seraient un cran au-dessus mais ajouteraient une dépendance native ; PBKDF2 est un compromis raisonnable et sans installation.*)

4.11 Streaming par Server-Sent Events plutôt que WebSocket ou polling

Choix : l'endpoint de chat répond en SSE (`text/event-stream`).

Un appel au tuteur peut durer plus d'une minute (recherche + génération). Un spinner muet est une mauvaise expérience. Le SSE diffuse la **progression réelle** : `planning` → `plan` → `step_start/step_done` par agent → `delta (tokens)` → `message final` → `title` → `done`. SSE est **unidirectionnel serveur→client** (exactement le besoin), **plus simple** qu'un WebSocket (pas de protocole bidirectionnel à gérer), et passe les proxys HTTP. `EventSource` ne supportant pas `POST`, le client parse lui-même le flux `fetch` (blocs séparés par une ligne vide).

4.12 Un « modèle rapide » (Haiku) pour les tâches utilitaires

Choix : `HERMES_FAST_MODEL = anthropic/claude-haiku-4-5` pour le planner, le titrage et l'indexation visuelle ; le modèle par défaut (plus grand) pour les agents qui *rédigent*.

Ces tâches ne produisent qu'un petit JSON, quelques mots de titre, ou une courte description d'image : un grand modèle y serait **surdimensionné** (latence $\times 3-4$ pour rien). Router ces appels vers un modèle rapide **divise la latence** et **le coût** sans dégrader la qualité perçue. Les agents qui rédigent (tuteur, IDP, contenu, compose) restent sur le modèle par défaut.

4.13 Vision pour indexer les schémas, avec cache et garde-fous

Choix : décrire les figures/schémas des cours par un modèle multimodal et indexer ces descriptions comme du texte recherchable.

Un schéma est invisible à un RAG purement textuel. On extrait les images **significatives** d'un PDF (PyMuPDF, filtre de taille pour ignorer logos/icônes, déduplication par [xref](#)), on les fait décrire par le modèle, et on indexe la description. Comme **un appel LLM par image coûte cher**, quatre garde-fous : filtre de taille ($\geq 300 \times 200$), **plafond** de 25 images/cours, **cache par hash** du contenu de l'image (réindexation gratuite), et **filtrage du décoratif** (le modèle répond **DECORATIF**, on n'indexe pas). L'enrichissement tourne **en tâche de fond** après l'upload : le texte du cours est disponible immédiatement, les figures arrivent ensuite.

4.14 React + FastAPI, l'interface Streamlit archivée

Choix : front React (Vite, TypeScript, Tailwind) sur une API FastAPI ; Streamlit conservé en archive de dépannage.

Streamlit a permis un prototypage rapide, mais son modèle « re-run intégral du script » plafonne pour une vraie expérience de chat (streaming fin, état riche, composants sur mesure). Le passage à React/FastAPI a été fait **sans réécrire le cœur** — preuve que la séparation en couches tenait. Les deux fronts ont coexisté le temps de la validation, d'où deux fichiers de conversations distincts pendant la transition (même schéma de message pour permettre une fusion simple).

5. Le moteur d'inférence : Hermes Agent

5.1 L'adaptateur `services/hermes_adapter.py`

Point d'entrée unique vers le modèle : `ask_hermes_with_skill(prompt, skill_name, timeout, max_attempts, backoff, model, on_delta)`. Il retourne un dictionnaire normalisé :

```
{"status": "success" | "error" | "not_available",
"content": str,           # réponse finale (ou "")
"method": "cli" | "not_available",
"error": None | str}
```

Construction de la commande (aucun shell) :

```
cmd = [hermes_bin, "-z", prompt]
if model:      cmd += ["-m", model]          # surcharge du modèle pour CET appel
if skill_name: cmd += ["--skills", skill_name] # None = appel neutre (titrage)
```

5.2 Robustesse des appels

Les échecs Hermes sont **intermittents** (timeout ponctuel, erreur transitoire du fournisseur, sortie vide). L'appel est donc :

- **réessayé** (`HERMES_MAX_ATTEMPTS = 2`, pause `HERMES_RETRY_BACKOFF_SECONDS = 2`) — sauf si le binaire est introuvable (un retry n'y changerait rien) ;
- **borné en temps** (`HERMES_TIMEOUT_SECONDS = 180`, calibré sur ~70 s d'appel normal + marge) ; au-delà, le processus est **tué** ;
- **journalisé** en cas d'échec dans un fichier réservé au développeur (`data/hermes_errors.log`), **jamais** montré à l'étudiant — un échec n'est donc plus « silencieux » ;
- **dégradé proprement** : si Hermes est injoignable, le prompt est **sauvegardé** (`data/last_tutor_prompt.md`) pour un test manuel, et le statut `not_available` remonte une explication claire.

Trois conditions d'échec sont distinguées à la lecture du résultat : code retour non nul, **sortie vide**, ou timeout. *(C'est précisément cette « sortie vide, exit 0 » qui, en exploitation, révèle un compte fournisseur sans crédits : l'API renvoie une erreur que la CLI avale, et l'adaptateur la classe en échec réessayable.)*

5.3 Le streaming *one-shot* (patch opt-in)

Par défaut, `subprocess.run` capture la sortie complète. Si `on_delta` est fourni, l'adaptateur bascule sur `_run_streaming`, qui exige le **patch Hermes** activé par `HERMES_ONESHOT_STREAM=1` (voir `docs/hermes-oneshot-streaming.patch`). Protocole : les tokens arrivent au fil de l'eau ; un **octet NUL** (`\x00`) marque une frontière de tour (le texte partiel est à jeter, `on_delta(None)` = remise à zéro) ; le **dernier segment** est la réponse canonique. Le décodage UTF-8 est **incrémental** (une séquence multi-octets coupée entre deux lectures est correctement recomposée), et le `select` respecte la **deadline** globale.

6. Le RAG, étage par étage

Pipeline complet : **document** → **segments structurés** → **chunks** → **embeddings** → **ChromaDB** → (**recherche** → **reranker**) → **contexte**, plus une branche **vision** parallèle.

6.1 Chargement structuré — `rag/document_loader.py`

Extrait des **segments** `{text, heading, page}`. Détection de titre multi-signaux (voir §4.7). Supporte `.md/.txt` (lecture directe) et `.pdf` (PyMuPDF avec tailles de police). Les formats Word/PowerPoint et l'OCR des scans sont pris en charge par le pipeline documentaire (dépendances `python-docx`, `python-pptx`, `pytesseract`).

6.2 Découpage — `rag/chunker.py`

`chunk_segments()` empaquette les segments à `CHUNK_TARGET_CHARS = 900` avec `CHUNK_OVERLAP_CHARS = 120`, en respectant paragraphes puis phrases, **sans jamais fusionner deux sections**. Chaque chunk porte : `id (<stem>_chunk_<n>)`, `source`, `chunk_index`, `text`, et si disponibles `title`, `section`, `page`.

6.3 Embeddings — `rag/embeddings.py`

Fonction d'embedding **partagée** (multilingue, cosinus), mise en cache après le premier chargement. Centralisée pour garantir la cohérence indexation/recherche.

6.4 Indexation — `rag/indexer.py`

Deux modes :

- `sync_courses_index()` — **incrémental** : compare les cours présents (et leur `mtime`) à ce qui est déjà indexé, puis n'agit que sur les différences (ajouté → indexé ; supprimé → retiré ; modifié → remplacé ; inchangé → **no-op**). Le rechargement d'une page ne réindexe donc rien. C'est le chemin normal (upload/suppression).
- `index_all_courses(reset=True)` — **rebuild complet** : reconstruit tout quand la *logique* d'indexation change (découpage, embedding). Astuce d'exploitation : la reconstruction se fait dans une collection **temporaire** puis **bascule** d'un coup, pour que la base active ne soit **jamais vide** pendant la (lente) vectorisation.

Les écritures ChromaDB sont **sérialisées** par un verrou (`_WRITE_LOCK`) car l'enrichissement vision tourne en arrière-plan.

6.5 Vision — `rag/vision_describe.py`

`course_vision_segments(path)` produit des segments `{text: "Figure (page N) : ...", heading: "Figure (schéma)", page: N}`, indexés comme du texte avec la métadonnée `kind="image"` et des `id` distincts (`<stem>_img_<n>`). `enrich_course_vision()` est **idempotent** (il supprime d'abord les anciennes figures du cours via un filtre `where`) et sérialisé. Garde-fous détaillés en §4.13.

6.6 Recherche et reclassement — `rag/retriever.py` + `rag/reranker.py`

- `search_course(query, n_results=20)` interroge ChromaDB et renvoie le vivier trié par distance.
- `rerank_chunks(query, chunks, top_k=5)` applique le cross-encoder (repli sur le tri par distance si indisponible).
- `list_indexed_courses()` liste les cours réellement indexés (l'agent « connaît » ses cours).
- `get_course_chunks(title)` reconstitue **tout** le texte d'un cours, ordonné, pour le résumé global (voir §7.6).

6.7 Mise en forme pour l'étudiant — `rag/source_formatter.py`

Transforme les chunks techniques en **indications lisibles** `{course, part, excerpt, document_path}`. Côté étudiant, on ne montre **jamais** distance, index de chunk ni top_k. La « partie » vient de la métadonnée `section` calculée à l'indexation (repli sur la page). Les extraits sont nettoyés (retrait du Markdown, du code, des tableaux) et tronqués proprement (60–90 caractères, coupe sur fin de phrase).

7. Le système multi-agents

7.1 Registre des capacités — `agents/registry.py`

Le « menu » déclaratif des agents (`tutor`, `idp`, `content`, `compose`), avec pour chacun : description, entrées, sorties, **préconditions**. Il sert (a) à informer le planner LLM des agents disponibles, et (b) de base à la **validation/réparation** côté Python (un `content` exige un `artifact` → sinon on insère un `idp`). Ajouter un agent = ajouter une entrée ici, plus son module et son skill.

7.2 L'orchestrateur — `agents/orchestrator.py`

Table d'intention déterministe (`_classify_to_plan`) :

Intention	Déclencheur (regex)	Plan d'agents
compose	verbe d'écriture + type de document	<code>compose</code> (ancrage cours optionnel)
analyze	« analyse », « structure », « de quoi ça parle »	<code>idp → content(summary)</code>
revision	« fiche », « révision », « flashcards »	<code>content(revision)</code>
summary	« résumé », « synthèse », « aperçu »	<code>content(summary)</code>
explain_section	« explique » + mot de section	<code>tutor</code> ancré sur la section
question (défaut)	tout le reste	<code>tutor</code>

Points remarquables :

- **Résolution du document cible** (`_match_doc`) : on ignore les tokens **partagés par tout le corpus** (ex. `cybersecurity`, `2026`) pour ne matcher que sur un token **distinctif** du nom de fichier — sinon un nom complet matcherait tous les cours (faux positif → clarification induite).

- **Réparation des préconditions** (`_repair_preconditions`) : insère un `idp` manquant avant un `content` (ou un `tutor` ancré sur section) si le document n'est pas encore analysé, sans doubler.
- **Itération sur le livrable courant** (`_is_revision_request`) : « raccourcis-le », « ajoute une section », « plus formel » **réutilisent** le dernier livrable au lieu de tout régénérer — mais uniquement si la consigne **vis** le livrable (verbe d'édition + cible : clitique `-le`, référence `ce document`, ou consigne « nue » comme « simplifie »). « Explique pourquoi on **ajoute** un pare-feu » n'est **pas** une révision.
- **Composition de la réponse finale** (`_compose`) : une réponse unique et lisible pour l'étudiant ; quand une ressource d'étude est produite, elle devient un **livrable structuré** (carte téléchargeable) et le chat n'affiche qu'une courte intro (pas de duplication du contenu).
- **Trace structurée** (`_trace`) réservée au développeur ; jamais montrée à l'étudiant.

7.3 Agent IDP (analyse documentaire) — `agents/idp_agent.py`

Pipeline : `extract_document` (déterministe) → contexte (texte intégral, ou **condensé** si trop long) → Hermes (skill `education-idp`) → **validation/ancrage** → `DocumentArtifact` (+ cache). La validation (`_validate_analysis`) ne garde que les `section_id` **réels** et jette les éléments mal formés. Si Hermes échoue ou rend un JSON inexploitable après 2 tentatives, l'artefact est conservé avec `analysis=None` : **jamais d'analyse à moitié fausse en silence**.

7.4 Agent de contenu — `agents/content_agent.py`

Transforme un document **déjà analysé** en `summary` ou `revision`. Il **ne contourne jamais l'IDP** : sans analyse, il renvoie `needs_analysis` (l'orchestrateur décidera). Il lit les sections **depuis l'artefact** (le contrat), peut cibler une **sélection de sections**, trace **déterministiquement** les sections utilisées, et **met en cache** par (`doc_id`, `type`, `options`).

7.5 Agent de rédaction (compose) — `agents/compose_agent.py`

Écrit un document **original** (rapport, exposé, note, synthèse...) à partir d'une consigne libre, façon *Artifacts/Canvas*. Contrairement à `content`, il **n'exige aucune source** (c'est un générateur, pas un transformateur) ; il peut **s'ancrer** sur un cours si l'étudiant en nomme un. `revise_document()` applique une consigne de modification et renvoie le document **complet** révisé. Cache par empreinte de (`consigne` + `contexte`).

7.6 Agent tuteur — `agents/tutor_agent.py`

Le cœur pédagogique. Orchestration : `question` → `recherche RAG` → `choix du mode` → `indications lisibles` → `prompt interne` → Hermes → `réponse structurée`.

Choix du mode pédagogique (`choose_tutor_mode`) selon la **distance du meilleur passage** :

Distance du meilleur chunk	Mode	Comportement
< 0.45	<code>course_grounded</code>	réponse ancrée sur le cours, structure en 5 points
< 0.65	<code>mixed</code>	base du cours + complément général, clairement séparés
≥ 0.65 ou rien	<code>general_tutor</code>	réponse générale honnête, sans fausse indication de cours

Autres finesses :

- **Requête de recherche adaptative** (`_build_retrieval_query`) : une question **auto-suffisante** est recherchée **seule** (top-k stable, indépendant du fil) ; une **relance elliptique** (« explique-le », « et Stuxnet ? ») réutilise les dernières questions de l'étudiant pour retrouver le sujet.
- **Conscience des cours** : si la recherche est faible mais que la question porte sur « le cours » en général, on s'appuie sur les cours réellement indexés (et on **demande lequel** s'il y en a plusieurs — réponse déterministe, sans appel LLM).
- **Résumé global** (`_answer_course_summary`) : pour « résume le cours », on envoie le **texte intégral** du cours en un seul appel (comme ChatGPT), avec un **garde-fou** de taille (`MAX_SUMMARY_INPUT_CHARS`) qui bascule sur un condensé (titres + amorces) au-delà.
- **Anti-injection** : le contenu de cours est encadré par des marqueurs « CONTENU DE COURS (non fiable) » et présenté comme **donnée**, jamais comme instruction ; les tentatives de changement de rôle dans le texte sont ignorées.
- **Séparation stricte** : `answer_student_question` renvoie tout (dont `debug` : chunks, prompt) ; `answer_student_question_for_ui` ne renvoie **que** les champs propres — jamais de chunk, distance ou prompt interne côté étudiant.

7.7 Titrage et présentation — `agents/titler.py`, `agents/presenter.py`

Le **titreur** produit un titre court (3–6 mots) du sujet via un appel Hermes **neutre** (sans skill, pour qu'aucune persona ne réponde au lieu de titrer), avec le **modèle rapide**. Best-effort : en cas d'échec, l'appelant garde son titre de repli. Le **présentateur** met en forme un artefact IDP pour l'affichage.

8. Multi-utilisateur et sécurité

8.1 Isolation totale par utilisateur

Chaque utilisateur possède un `user_id` opaque (`u_<hex>`) qui **cloisonne toutes ses données** :

```
data/users/<uid>/courses/      ses cours
data/users/<uid>/artifacts/      ses artefacts IDP
data/users/<uid>/generated/      ses contenus générés
data/users/<uid>/attachments/    ses pièces jointes
data/users/<uid>/conversations.json son historique
+ collection ChromaDB course_chunks_<uid> ses vecteurs
```

L'**isolation vectorielle est par construction** : une **collection ChromaDB distincte par utilisateur** rend **impossible** la fuite des cours d'un autre par un filtre oublié — contrairement à une collection partagée filtrée par `user_id`, où un oubli de filtre exposerait tout le monde.

8.2 Le contexte utilisateur — `config/workspace.py`

Le `user_id` courant vit dans une `contextvars.ContextVar` (défaut `u_salah`, le compte historique). Un middleware la pose au début de chaque requête ; les couches profondes (RAG, caches, store) résolvent le bon workspace via des helpers (`courses_dir()`, `collection_name()`, ...) **sans jamais recevoir `user_id` en paramètre**. Cela évite de fil-de-fériser `user_id` à travers toute la pile.

Piège des threads, résolu : une `ContextVar` **ne se propage pas** aux threads créés manuellement. Le worker SSE et le thread de titrage capturent donc le contexte (`ctx = contextvars.copy_context()`) et s'exécutent dedans (`ctx.run(...)`). Le générateur SSE, qui peut tourner hors requête, repose aussi `set_current_user(user_id)` par sécurité.

8.3 Authentification — `api/auth.py`

Comptes `identifiant + mot de passe`, hachés PBKDF2 (§4.10), persistés dans `data/users.json` (§600). Tokens HMAC *stateless* (§4.9). `ensure_default_user()` crée au démarrage le compte propriétaire des données pré-existantes (`salahoudine934@gmail.com`, id `u_salah`).

8.4 Autorisation — `api/main.py`

Double barrière :

- un **middleware** lit le token et pose le contexte utilisateur (sans bloquer) ;
- une **dépendance** `Depends(current_user)` sur chaque endpoint protégé renvoie **401** si le token est absent/invalidé.

Protection de chemin : les accès fichiers utilisent `Path(filename).name` pour empêcher la **traversée de répertoire** (`../..`).

9. Le backend API (FastAPI + SSE) — `api/main.py`

Endpoints principaux :

Méthode & route	Rôle
<code>POST /api/auth/register · login · GET /me</code>	comptes et session
<code>GET /api/health</code>	statut + disponibilité de l'export PDF
<code>GET/POST/DELETE /api/documents...</code>	cours (liste, upload+indexation, suppression, fichier)
<code>POST /api/attachments · GET .../file</code>	pièces jointes du chat (hors RAG)
<code>GET /api/deliverables</code>	Bibliothèque (livrables + pièces jointes)
<code>GET/POST/DELETE /api/conversations...</code>	historique
<code>POST /api/conversations/{id}/messages</code>	chat en SSE
<code>POST /api/export</code>	export d'un livrable en <code>docx/md/pdf</code>

Le chat en SSE (`post_message`) est le morceau le plus subtil : il persiste le message utilisateur, lance en **parallèle** le titrage (premier échange) et le worker de l'orchestrateur (chacun dans le **contexte capturé**), et **stream** les événements via une `queue.Queue` que le générateur draine vers le client. La réponse finale est persistée (même schéma que Streamlit), puis le titre, puis `done`.

Pièces jointes vs cours : une pièce jointe est un fichier **ponctuel** attaché à un message (pas indexé au RAG). Les **images** sont transmises au tuteur par **chemin** (`image_path`) pour être **vues** nativement par le modèle multimodal (schémas compris, pas seulement l'OCR) ; les autres formats sont injectés en **texte extrait**, encadrés comme donnée non fiable.

10. Le frontend (React) — `webapp/`

Stack : **React + Vite + TypeScript + Tailwind**. Organisation :

- `lib/session.ts` : `getToken/setToken/clearToken`, `authFetch` (un **401** déclenche déconnexion), et `openAuthed` (télécharge un fichier protégé via `fetch → blob → window.open`, car `window.open` ne peut pas porter d'en-tête d'autorisation) ;
- `lib/api.ts` : tous les appels REST, typés, via `authFetch` ;
- `lib/chat.ts` : **client SSE** maison (parse le flux `fetch`, blocs `data: {json}` séparés par ligne vide) et téléchargement des livrables ;
- `App.tsx` : machine à états de session (`undefined` = vérification, `null` = déconnecté, `User` = connecté), chargement des données après connexion, gestion du flux de progression (`planning/plan/step_*/delta/message/title`) ;
- composants : `Login`, `Sidebar`, `ChatView`, `Message`, `Home`, `Courses`, `MediaLibrary`, `DeliverableCard`, `ProgressTimeline`, `PromptBar`.

11. Persistance, identité et caches — `agents/document_store.py`, `api/store.py`

- **Identité par contenu** : `doc_id = SHA-256(contenu)[:16]` (pas le nom). Détecte modification et doublon, et donne une identité **stable** partagée par le RAG, l'IDP et l'agent de contenu.
- **Artefacts IDP** : `data/users/<uid>/artifacts/<doc_id>.json`, invalidés automatiquement si `ARTIFACT_SCHEMA_VERSION` change (un bump force la réanalyse).
- **Contenus générés** : `.../generated/<doc_id>__<type>__<hash-options>.json` → on ne rappelle pas Hermes pour une génération identique.
- **Conversations** : `.../conversations.json`, accès sérialisé, plafond de 200 (purge des plus anciennes par activité).
- **Lignée de livrables** : chaque livrable porte un `id` de **lignée** et une `version`. Une révision **prolonge** la lignée (même `id`, `version+1`). La Bibliothèque **regroupe** par lignée et n'affiche que la **dernière** version (avec `version_count`) — pas de doublons v1/v2/v3.

12. Robustesse et gestion d'erreurs

Le système est conçu pour **ne jamais casser en silence** et pour **dégrader proprement** :

- appels Hermes : retries, timeout, journal développeur, sauvegarde du prompt, statut `not_available` explicite (§5.2) ;
- reranker indisponible → repli sur le tri par distance ;
- vision indisponible → le texte du cours reste indexé (best-effort) ;
- IDP échoué → `analysis=None` (jamais d'analyse fausse) ;
- persistance JSON → écritures best-effort, jamais bloquantes ;
- fournisseur LLM en panne → le routage déterministe, le calcul du mode et les indications de cours **fonctionnent quand même** ; seul le texte final manque, avec un message clair.

13. Stratégie de test — `tests/`

La suite `pytest` est **déterministe et sans appel LLM** (les appels Hermes sont mockés), donc rapide et reproductible en CI :

- `test_auth.py` : hachage, tokens (round-trip, altération), création/authentification ;
- `test_isolation.py` : cloisonnement multi-utilisateur (workspaces, collections) ;
- `test_rag_units.py`, `test_vision_rag.py` : découpage, indexation, vision ;
- `test_orchestrator_routing.py`, `test_tutor_logic.py`, `test_agents_units.py` : routage déterministe, modes, agents ;
- `test_api_endpoints.py` : endpoints (avec utilisateur+token et titrage mocké) ;
- `test_exporters.py`, `test_hermes_adapter.py` : export et adaptateur.

Une *fixture* d'autoreset de la `ContextVar` (`conftest.py`) évite les fuites de contexte entre tests.

14. Parcours d'une requête, de bout en bout

« Explique-moi la cyber kill chain » sur un compte connecté :

1. **Front** : `streamMessage POST /api/conversations/{id}/messages` avec le token Bearer.
2. **API** : middleware → contexte utilisateur ; `Depends(current_user)` → 200 ; persistance du message ; capture du contexte ; lancement du worker et (1er échange) du titreur.
3. **Orchestrateur** : `_classify_to_plan` → intention `question` → plan `[tutor]`.
4. **Tuteur** : `_build_retrieval_query` (question auto-suffisante → recherche seule) → `search_course` (20 candidats) → `choose_tutor_mode` (distance < 0.45 → **course grounded**) → `rerank_chunks` (5 meilleurs) → `format_course_indications` → `build_tutor_prompt`.
5. **Hermes** : `hermes -z <prompt> --skills education-tutor`, **streamé** ; chaque token devient un événement SSE `delta`.

6. **API** : réponse finale persistée et émise (`message`), puis `title`, puis `done`.
7. **Front** : la timeline montre `planning` → `tutor`, le texte s'écrit en direct, la réponse structurée s'affiche (5 points), la sidebar reçoit son titre.

15. Référence fichier par fichier

`config/` - `settings.py` — configuration centrale, **aucune clé API**, tout surchargeable par variable d'environnement (chemins, RAG, seuils, Hermes). - `workspace.py` — espace de travail par utilisateur (ContextVar + helpers de chemins et de nom de collection).

`services/` - `hermes_adapter.py` — appel Hermes CLI (one-shot, streaming, retries, journal). - `exporters.py` — export Markdown → `docx` (python-docx) / `md` / `pdf` (markdown → HTML → xhtml2pdf).

`rag/` - `document_loader.py` — extraction structurée (segments, détection de titres multi-signaux). - `chunker.py` — découpage structure-aware. - `embeddings.py` — fonction d'embedding multilingue partagée. - `indexer.py` — indexation incrémentale + rebuild + enrichissement vision. - `vision_describe.py` — description des figures par vision (cache, garde-fous). - `reranker.py` — cross-encoder (repli propre). - `retriever.py` — recherche, reclassement, liste des cours, texte intégral. - `source_formatter.py` — indications de cours lisibles pour l'étudiant.

`agents/` - `registry.py` — menu déclaratif des capacités. - `orchestrator.py` — routage déterministe, réparation, exécution, composition, trace. - `artifact.py` — contrat `DocumentArtifact` (extraction déterministe + analyse ancrée). - `document_store.py` — `doc_id` par contenu, caches artefacts/générés. - `idp_agent.py` — analyse documentaire (validation, ancrage, cache). - `content_agent.py` — résumé / fiche de révision (jamais sans IDP). - `compose_agent.py` — rédaction et révision de documents originaux. - `tutor_agent.py` — cœur pédagogique (modes, requête adaptative, résumé global, anti-injection). - `titler.py` — titre de conversation (Hermes neutre, modèle rapide). - `presenter.py` — mise en forme d'un artefact IDP. - `common.py` — utilitaires partagés (extraction JSON tolérante).

`api/` - `main.py` — façade HTTP (auth, contexte, endpoints, SSE, export). - `auth.py` — comptes, hachage, tokens, compte par défaut. - `store.py` — persistance des conversations et Bibliothèque (lignée de livrables).

`webapp/` — front React (voir §10). `scripts/migrate_to_multiuser.py` — migration idempotente vers le multi-utilisateur. `archive/streamlit_app.py` — ancienne UI (dépannage).

16. Limites connues et évolutions

- **Mono-serveur** : persistance JSON + verrous process-locaux ; une montée en charge multi-process nécessiterait une base transactionnelle et un stockage vectoriel partagé.
- **Révocation de session** : pas de révocation fine (changer le secret invalide tout).

- **Dépendance au fournisseur LLM** : sans crédits/clé valide, les fonctions génératives sont indisponibles (le reste fonctionne). Une bascule de fournisseur (ex. OpenRouter) est possible côté configuration Hermes.
- **Capacités Hermes non encore exploitées** : exécution de code et recherche web ([execute_code](#), [web_search](#)) sont identifiées comme évolutions, à intégrer en posture de tuteur.
- **Qualité RAG** : les hyperparamètres (modèle d'embedding, [n_results](#), reranker) sont volontairement laissés en l'état ; à ajuster seulement si la performance observée le justifie.

17. Glossaire

- **RAG** — *Retrieval-Augmented Generation* : on récupère des passages pertinents et on les fournit au modèle pour ancrer sa réponse.
- **Chunk** — fragment de cours indexé (texte + métadonnées).
- **Embedding** — vecteur numérique représentant le sens d'un texte.
- **Bi-encoder / Cross-encoder** — encodage séparé (rapide) vs conjoint question×passage (précis).
- **Skill Hermes** — module de comportement (persona, format, garde-fous) chargé via [--skills](#).
- **DocumentArtifact** — contrat de données commun (extraction déterministe + analyse ancrée).
- **Livable** — ressource produite (résumé, fiche, document), téléchargeable, versionnée par lignée.
- **SSE** — *Server-Sent Events* : flux serveur→client pour la progression temps réel.
- **ContextVar** — variable de contexte Python portant l'utilisateur courant à travers la pile.

Document généré à partir du code source d'EduTutor (branche [main](#)). Chaîne de génération :
Markdown → HTML → PDF via l'exporteur de l'application ([services/exporters.py](#), [xhtml2pdf](#)).